C:\PROYECTOS**DATATUTOR>**

_

[DOCUMENTO TÉCNICO]

Nombre del proyecto

DataTutor

Tutor RAG para bases de datos, ingeniería y ciencia de datos

Agente LangChain + LangGraph · modelo local · pgvector

// INTEGRANTES

Michelle Ruiz Segura
Juan José Ramírez Pulgarín
Isaac Camilo Giraldo Gómez

Integración e Implementación de Software // Octubre 2026

// Contenido

1. Nombre del proyecto e integrantes
2. Tema seleccionado y descripción del problema
3. Arquitectura general y diagrama de arquitectura
4. Tecnologías utilizadas
5. Descripción backend
6. Descripción frontend
7. Descripción del agente y flujo LanGraph
8. Modelo local utilizado y base de datos vectorial
9. Proceso de carga de documentos
10. Estrategías implementadas para evitar alucinaciones
11. Seguridad y autenticación
12. Despliegue
13. Evidencia de funcionamiento
14. Problemas encontrados y conclusiones

#01 Nombre del proyecto e integrantes

DataTutor es un tutor basado en RAG (Retrieval-Augmented Generation) para bases de datos, ingeniería de datos y ciencia de datos. Responde únicamente con lo que respalda su base de conocimiento, cita el documento y la página de donde sale cada respuesta y rechaza de forma explícita lo que no puede sustentar.

> Integrantes

- Michelle Ruiz Segura
- Juan José Ramírez Pulgarín
- Isaac Camilo Giraldo Gómez

#02 Tema seleccionado y descripción del problema

> Tema

Asistente educativo para consultar libros y artículos de **bases de datos**, **ingeniería de datos** y **ciencia de datos** mediante preguntas en lenguaje natural. El conocimiento vive en una base vectorial propia; el modelo de lenguaje solo redacta a partir de lo que se recupera de ella.

> Problema

Quien estudia estas áreas trabaja con material muy extenso (el libro de Silberschatz, por ejemplo, tiene 673 páginas) y en dos idiomas. Encontrar un concepto concreto, como las formas normales, obliga a recorrer índices y capítulos. Los asistentes de IA generales ayudan, pero tienen dos defectos serios para un contexto académico: **pueden inventar respuestas con apariencia de verdad** y **no indican en qué fuente se apoyan**.

DataTutor busca resolver ese problema con un asistente que sea verificable: cada respuesta debe poder rastrearse hasta un documento y una página, y cuando el material cargado no alcanza, el sistema debe decirlo en lugar de improvisar.

> **Mensaje de rechazo obligatorio.** Cuando no hay información suficiente, el agente responde: «No tengo suficiente información en mi base de conocimiento para responder esa pregunta. Te sugiero cargar documentos relacionados con el tema.»

#03 Arquitectura general y diagrama de arquitectura

DataTutor sigue una arquitectura de tres capas más servicios de apoyo.

1) El navegador ejecuta el frontend en React, que autentica al usuario con Keycloak y envía a la API cada petición con un token JWT.
2) La API FastAPI valida el token y delega en servicios; estos invocan al agente LangGraph o a los repositorios.
3) El agente usa Ollama para el modelo de chat y los embeddings, y PostgreSQL con *pgvector* como base vectorial y de historial.

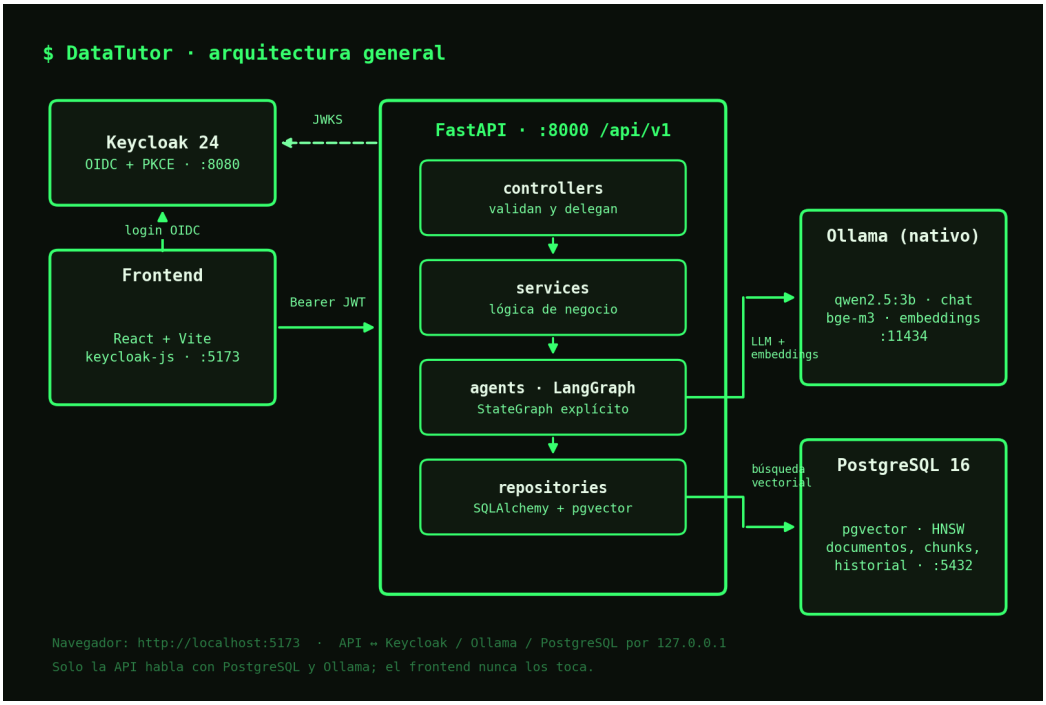


Figura 1. Arquitectura general de DataTutor

> Dónde corre cada pieza

Pieza	Dónde corre	Dirección
Frontend (React + Vite)	Servidor de desarrollo de Vite en el portátil	`localhost:5173`
Keycloak 24	Contenedor Docker	`localhost:8080`
API FastAPI	Directamente en el host (PyCharm o uvicorn)	`127.0.0.1:8000`
PostgreSQL 16 + pgvector	Contenedor Docker, volumen `datatutor-pgdata`	`127.0.0.1:5432`
Ollama	Nativo en Windows, solo CPU	`127.0.0.1:11434`

> Patrones aplicados

Patrón	Dónde	Beneficio
Capas controller → service → repository	Todo el backend	Pruebas aisladas y responsabilidades claras
Inyección de dependencias (`Depends`)	`core/deps.py`	Sustituir LLM, retriever o repositorio por dobles de prueba
Adapter	`vectorstore/retriever.py`	El grafo no se acopla a pgvector
Factory	`llm/`	Cambiar de modelo por variable de entorno
Máquina de estados	`agents/graph.py`	Flujo explícito y explicable del agente
Idempotencia por hash	`services/ingestion.py`	Reintentar la carga sin duplicar documentos

#04 Tecnologías utilizadas

La base del sistema es **LangChain**, **LangGraph** y la arquitectura **RAG**. Alrededor de ellas se eligieron herramientas gratuitas que funcionan en una sola máquina sin GPU. Las versiones corresponden a las registradas en el proyecto (`uv pip list` y `npm list`).

Capa	Tecnología	Versión	Rol en DataTutor
Agente	LangChain	1.4.3	Integración con el modelo y los embeddings
Agente	LangGraph	1.2.14	`StateGraph` con nodos y aristas explícitos
Agente	langchain-ollama	1.1.0	Conexión con el modelo de chat y los embeddings
Backend	Python	3.12.7	Lenguaje, gestionado con `uv`
Backend	FastAPI / uvicorn	0.142.2 / 0.54.0	API REST y servidor ASGI
Backend	Pydantic / pydantic-settings	2.13.5 / 2.15.0	Esquemas y configuración tipada
Datos	SQLAlchemy / psycopg / pgvector	2.1.3 / 3.3.6 / 0.5.0	ORM, controlador y tipo vectorial
Datos	PostgreSQL 16 + pgvector	Docker	Base relacional y vectorial (índice HNSW)
Modelo local	Ollama · `qwen2.5:3b`	CPU	Clasificar, juzgar, reescribir y responder
Modelo local	Ollama · `bge-m3`	1024 dim.	Embeddings de fragmentos y preguntas
Documentos	PyMuPDF	1.28.2	Extracción de texto por página
Seguridad	Keycloak / PyJWT / cryptography	24 / 2.15.1 / 50.0.2	OIDC + PKCE y validación de JWT RS256
Frontend	React / react-dom	19.3.1	Interfaz de usuario
Frontend	Vite	8.3.4	Servidor de desarrollo y empaquetado
Frontend	keycloak-js	26.2.4	Inicio de sesión OIDC con PKCE
Calidad	pytest / ruff / Oxlint	9.1.1 / 0.16.10 / 1.88.0	Pruebas y análisis estático
Infraestructura	Docker Compose	v5.6.0	PostgreSQL y Keycloak reproducibles

> **Restricción cumplida.** El flujo del agente está escrito en código con LangChain y LangGraph; no se usó ninguna herramienta visual o de arrastrar y soltar.

#05 Descripción del backend

El backend es una API REST en **FastAPI** (Python 3.12) con prefijo `/api/v1`. Todas las rutas, excepto `GET /health`, exigen un token Bearer emitido por Keycloak. El código se organiza en capas para que cada pieza pueda probarse por separado.

> Estructura

```
backend/app/
  main.py          crea la app, registra routers y manejadores de error
  config/          settings.py (pydantic-settings)
  core/            database.py, deps.py, errors.py, security.py (JWT)
  controllers/     chat.py, documents.py   → solo validan y delegan
  schemas/         modelos Pydantic de entrada y salida
  services/        chat.py, documents.py, ingestion.py
  repositories/    models.py, chat_repo.py, document_repo.py
  agents/          graph.py, state.py, prompts.py, nodes/core.py
  vectorstore/     retriever.py (adaptador sobre repositories)
  llm/             chat.py, embeddings.py (fábricas de Ollama)
backend/tests/     test_api_contracts.py
backend/eval/      questions.jsonl   · backend/scripts/  ingest_corpus.py, ...
```

> Endpoints

Método y ruta	Función	Códigos
`GET /health`	Estado del servicio (única ruta pública)	200
`GET /me`	Perfil del token: `sub`, `preferred_username`, roles	200, 401
`POST /chat`	Un turno del agente: `{question, conversation_id?}` → respuesta, decisión, fuentes y latencia	200, 401, 422, 503
`GET /conversations`	Conversaciones del usuario, las recientes primero	200
`GET /conversations/{id}/messages`	Historial de una conversación	200, 404
`DELETE /conversations/{id}`	Borra la conversación y sus mensajes (cascada)	204, 404
`POST /documents`	Sube un archivo (multipart) y lo procesa en segundo plano	202, 409, 413, 415
`GET /documents`	Documentos propios y globales con su estado	200
`GET /documents/{id}`	Detalle de un documento	200, 404
`DELETE /documents/{id}`	Borra un documento propio y sus fragmentos	204, 404

> Convenciones de la API

» **Un rechazo del agente es HTTP 200** con `decisión = "rejected"`: es una respuesta válida del sistema, no un error.

- **Formato único de errores:** `{ "error": { "code", "message", "request_id" } }`. Las excepciones de dominio se convierten a HTTP en un único manejador.
- Una conversación o documento ajeno responde **404**, para no revelar que existe.
- Si el modelo o la base de datos no responden, la API devuelve **503** e incluye el `conversation_id` para que el frontend no cree conversaciones duplicadas al reintentar.
- Al arrancar, `fail_interrupted_documents` marca como fallidos los documentos que quedaron en `pending` o `processing` por un reinicio.
- La pregunta debe tener entre 2 y 2000 caracteres; una pregunta en blanco devuelve **422**.

> Limitación conocida

El envío de mensajes no es idempotente: si se reintenta la misma pregunta tras un 503, se reutiliza la conversación pero se inserta otro mensaje de usuario.

#06 Descripción del frontend

El frontend es una aplicación **React 19** con **Vite** y **keycloak-js**, de una sola página. Está pensado para usarse siempre desde `http://localhost:5173`, el único origen permitido por el cliente de Keycloak.

> Funciones

- **Inicio de sesión obligatorio:** la aplicación redirige a Keycloak (`login-required`) y no muestra nada a usuarios sin autenticar. Incluye registro y cierre de sesión.
- **Chat con el agente:** el usuario escribe una pregunta y recibe la respuesta junto con su latencia y sus fuentes (documento, página, similitud y fragmento).
- **Insignias por decisión:** `answered` en verde, `clarify` en azul, `rejected` en rojo o ámbar y `smalltalk` en gris, para distinguir de un vistazo cómo respondió el agente.
- **Historial:** barra lateral con las conversaciones del usuario, que se pueden abrir o borrar.
- **Documentos:** subida de PDF, lista con estado (`pending`, `processing`, `ready`, `failed`) y botón de borrado solo para documentos propios.
- **Errores visibles:** un banner informa cuando el agente no puede responder o la API falla, tal como exige el enunciado.

> Detalles de implementación

Archivo	Responsabilidad
`services/auth.js`	Instancia única de `keycloak-js` (`initPromise` evita el doble `init()` de `React.StrictMode`). Usa PKCE S256 y renueva el token con `updateToken(30)` antes de cada petición. El token vive en memoria, no en `localStorage`.
`services/api.js`	Adjunta el `Bearer`, no fija `Content-Type` cuando el cuerpo es `FormData` (el navegador añade el `boundary`) y traduce el formato de error del backend.
`App.jsx`	Barra lateral, área de chat, banner de errores y consulta periódica (cada 3 s) de `GET /documents` mientras haya documentos en proceso.
`styles.css`	Estilos de la interfaz.

El proyecto compila sin errores con `vite build` y se revisa con Oxlint. Las variables **públicas** usan el prefijo `VITE_` y no incluyen ningún secreto.

#07 Descripción del agente y del flujo de LangGraph

El agente se construye en código con un `StateGraph` explícito. No se usa un agente React genérico, porque el enunciado exige nodos propios de **validación**, **evaluación** de suficiencia y **rechazo**.

Cada turno recorre el grafo de la Figura 2 y termina siempre en `save_history`.

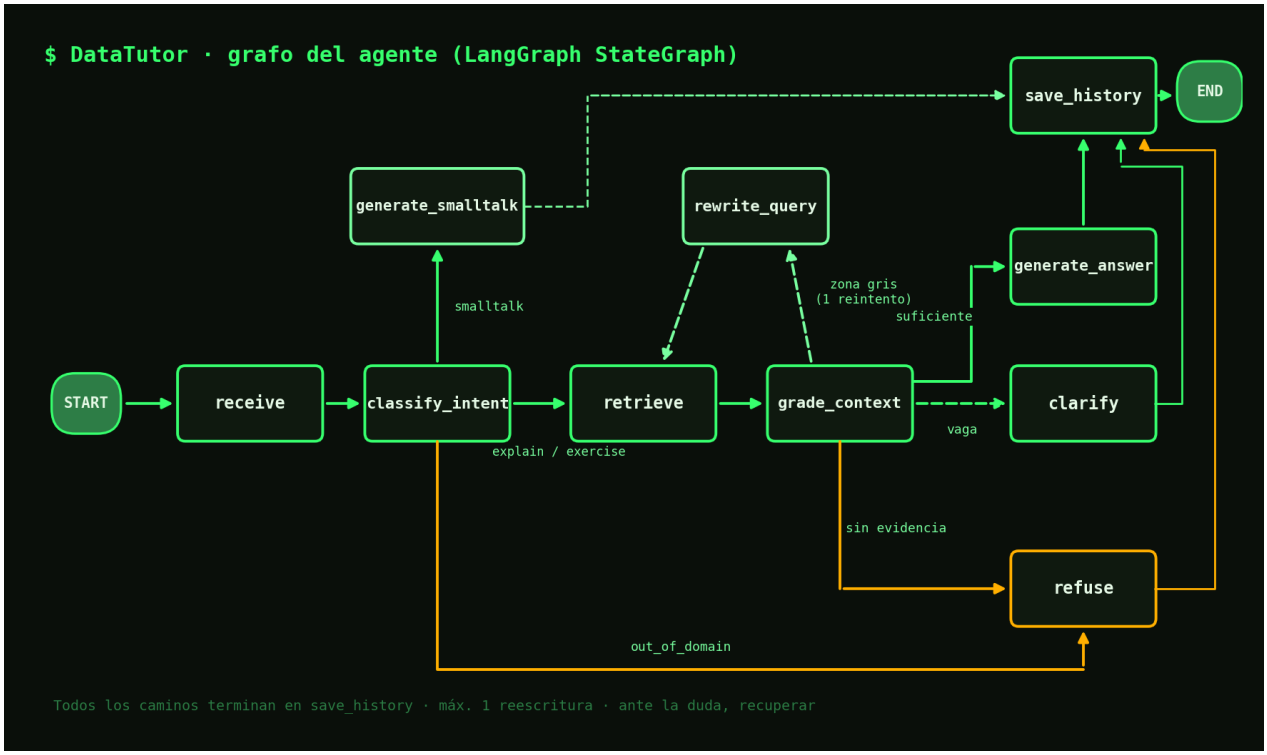


Figura 2. Flujo del agente en LangGraph

> Nodos

Nodo	Qué hace
`receive`	Normaliza la pregunta, recorta el historial (últimos 6 turnos) y valida la longitud.
`classify_intent`	Clasifica la intención: `explain`, `exercise`, `smalltalk` o `out_of_domain`. Ante la duda deja pasar: un falso «fuera de dominio» es peor que recuperar de más.
`generate_smalltalk`	Responde saludos y conversación trivial sin consultar la base.
`retrieve`	Genera el embedding de la consulta y busca los 4 fragmentos más cercanos con pgvector, filtrando por `owner_id` ∈ {`global`, `sub`}. Puntaje = 1 − distancia coseno.
`grade_context`	Nodo anti-alucinación: decide si lo recuperado alcanza para responder (ver más abajo).
`rewrite_query`	En la zona gris reescribe la consulta una sola vez (por ejemplo, traducirla) y vuelve a `retrieve`.
`generate_answer`	Redacta la respuesta con un prompt restrictivo que obliga a usar solo el contexto recuperado.
`clarify`	Pide precisar la pregunta cuando es muy corta o ambigua (4 palabras o menos).

Nodo	Qué hace
`refuse`	Devuelve el mensaje fijo de rechazo y sugiere cargar documentos.
`save_history`	Cierra el turno. La persistencia real del mensaje y su metadato (`decision`, `sources`, `best_score`, `latency_ms`) ocurre en `services/chat.py`.

> Decisión de suficiencia en grade_context

Condición	Comportamiento
`best_score` < 0,50 (`SIM_LOW`)	Contexto insuficiente; no se llama al juez.
0,50 ≤ `best_score` < 0,65, primer intento	Insuficiente; se reescribe la consulta y se recupera de nuevo.
0,50 ≤ `best_score` < 0,65, tras reescribir	Evalúa el juez LLM.
`best_score` ≥ 0,65 (`SIM_THRESHOLD`)	Evalúa el juez LLM; si no responde `true`, entra el respaldo léxico.

El juez es el propio modelo local. Lee la pregunta y los fragmentos recuperados y contesta con una salida estructurada: {sufficient: true} si el material alcanza para responder o {sufficient: false} si no. Como el modelo es pequeño, a veces no contesta en el formato correcto. Cuando eso ocurre (un JSON inválido o una respuesta que no sea verdadero o falso), el sistema lo cuenta siempre como «no suficiente» y nunca como éxito: ante la duda, es preferible no responder a arriesgarse a inventar.

Respaldo léxico

El problema. En las pruebas, la pregunta sobre normalización recuperó fragmentos muy relevantes del libro de Silberschatz (puntaje de similitud de 0,706, que es alto). Aun así, el juez respondió «no suficiente» por error. Es un falso negativo: había información buena y el sistema la rechazó. Cambiar las instrucciones del juez no lo corrigió.

La solución. Se añadió un segundo chequeo, mucho más simple, que actúa solo cuando el juez dijo «no». En lugar de confiar en el modelo, compara palabras entre la pregunta y los fragmentos:

1. Toma las palabras importantes de la pregunta: las de 5 letras o más, sin palabras comunes (como «para») ni términos demasiado genéricos (como «datos»).
2. Cuenta cuántas de ellas aparecen en los fragmentos recuperados.
3. Acepta el contexto solo si coinciden al menos 2 palabras, esas palabras son al menos el 35 % de las importantes de la pregunta y el puntaje de similitud es mayor o igual a 0,65 (SIM_THRESHOLD).

Ejemplo

Pregunta: «¿Qué es la normalización y para qué sirven las formas normales?»

- Palabras importantes de la pregunta: normalización, sirven, formas, normales (4 palabras).

- Palabras que aparecen en los fragmentos: normalización, formas, normales (3 de 4).
- Proporción de coincidencia: 3 de 4 = 75 %, más que el 35 % exigido y con más de 2 palabras.
- Puntaje de similitud: 0,706, mayor que 0,65.
- Decisión del respaldo: se acepta el contexto y el agente responde, aunque el juez se haya equivocado.

Limitaciones del respaldo. Solo compara palabras exactas y no entiende el significado, por lo que puede aceptar contexto débil si comparte términos con la pregunta. Tampoco ayuda cuando la pregunta está en español y los fragmentos en inglés, y no rescata contextos con puntaje por debajo del umbral. Por eso exige varias coincidencias y un puntaje alto.

> **Limitaciones del respaldo.** Es una heurística: puede aceptar contexto débil con términos coincidentes, rara vez ayuda cuando la pregunta está en español y los fragmentos en inglés y no rescata contextos por debajo del umbral.

> Parámetros del agente

Parámetro	Valor	Parámetro	Valor
TOP_K	4	temperature	0,0
SIM_THRESHOLD	0,65	num_ctx	4096
SIM_LOW	0,50	num_predict	512
CHUNK_SIZE / CHUNK_OVERLAP	1000 / 200	Reescrituras máximas	1

El agente se prueba con un modelo y un recuperador falsos (inyección de dependencias), de modo que las rutas de respuesta, rechazo, reescritura y aclaración pueden verificarse sin ejecutar el LLM.

#08 Modelo local utilizado y base de datos vectorial

> Modelo de lenguaje

DataTutor usa **'qwen2.5:3b'** a través de **Ollama**, ejecutado de forma nativa en Windows y solo con CPU. El mismo modelo clasifica la intención, juzga la suficiencia del contexto, reescribe consultas y redacta la respuesta.

Pregunta	Respuesta
¿Por qué este modelo?	El equipo tiene 16 GB de RAM y no usa GPU (la GPU del equipo está dañada y Ollama falla en ella). Un modelo de 3B ocupa unos 2–3 GB, deja margen para Keycloak, PostgreSQL y el IDE, y es gratuito.

Pregunta	Respuesta
¿Cómo se ejecuta?	Con `ollama serve` forzando CPU (ver bloque inferior). La API lo consulta por `127.0.0.1:11434` mediante `langchain-ollama`.
¿Con qué parámetros?	`temperature` 0,0 para respuestas deterministas, `num_ctx` 4096 y `num_predict` 512.
¿Qué limitaciones tiene?	Latencia de 12 a 33 s por consulta y un juez de suficiencia poco fiable, que motivó el respaldo léxico. Además, un modelo pequeño puede apoyarse en conocimiento propio aunque se le pida usar solo el contexto.

```
$env:CUDA_VISIBLE_DEVICES="-1"
$env:OLLAMA_LLM_LIBRARY="cpu_avx2"
ollama serve
```

> Modelo de embeddings

Los fragmentos y las preguntas se convierten en vectores con **'bge-m3`** (1024 dimensiones), también servido por Ollama. Se eligió porque el corpus **mezcla** español e inglés. Cambiar de modelo de embeddings implica modificar la columna vectorial y volver a generar todos los vectores.

> Base de datos vectorial: PostgreSQL + pgvector

Una sola base de datos guarda los **metadatos**, los fragmentos con sus **vectores** y el **historial**. Corre en Docker con un volumen persistente (`datatutor-pgdata`), de modo que los embeddings sobreviven a los reinicios. El acceso se hace con **SQLAlchemy** a través de repositorios.

Tabla	Contenido principal
`documents`	Metadatos del archivo: propietario, `scope` (`user` o `global`), nombre, hash SHA-256, páginas, número de fragmentos, estado y error.
`chunks`	Texto del fragmento, página, `embedding vector(1024)`, modelo de embeddings, `owner_id` y referencia al documento (`ON DELETE CASCADE`).
`conversations`	Conversaciones por usuario.
`messages`	Mensajes con rol, contenido, `decision`, fuentes (JSONB), `best_score` y `latency_ms` (`ON DELETE CASCADE` desde la conversación).

- **Índice HNSW** sobre `embedding` con distancia coseno (`vector_cosine_ops`) para acelerar la búsqueda.
- **Restricción única** `(owner_id, sha256)`: el mismo archivo no se duplica por propietario.
- **Aislamiento:** cada búsqueda filtra por `owner_id IN ('global', <sub del token>)`.
- **Puntaje:** como pgvector devuelve distancia coseno, el puntaje es `1 − distancia`; así 1 es idéntico.

> Lección sobre los embeddings

El campo `embed_model = 'bge-m3'` no prueba que el vector se haya generado con ese modelo. Los 57 primeros fragmentos tenían un coseno de −0,056 contra una regeneración fresca; tras regenerarlos en sitio el coseno fue 1,000000. Por eso, después de una ingesta grande se valida una muestra (coseno almacenado frente a regenerado, sin nulos ni normas cero).

#09 Proceso de carga de documentos

La carga convierte un archivo en conocimiento consultable. Es **asíncrona**: la API responde de inmediato y el procesamiento ocurre en segundo plano con `BackgroundTasks`.

> Flujo

1. **Recepción.** El usuario envía el archivo a `POST /documents` (multipart).
2. **Validación.** Se comprueban el tamaño (máximo 30 MB), la extensión y el tipo real del archivo, y se calcula el hash SHA-256. Si ya existe para ese usuario, la API responde 409 sin reprocesar.
3. **Registro.** Se crea el documento en estado `pending` y se responde **202** con su identificador.
4. **Extracción.** En segundo plano (`processing`), PyMuPDF extrae el texto página por página.
5. **Fragmentación.** Cada página se divide en fragmentos de 1000 caracteres con 200 de solape.
6. **Embeddings.** `bge-m3` genera un vector de 1024 dimensiones por fragmento.
7. **Almacenamiento.** Los fragmentos se insertan en lotes en `chunks`, asociados al documento original, con su página y su propietario.
8. **Confirmación.** El documento pasa a `ready` y el frontend lo muestra como disponible.

> Estados de un documento

Estado	Significado
`pending`	Recibido y registrado; en cola.
`processing`	Extrayendo, fragmentando y generando embeddings.
`ready`	Procesado con éxito y consultable.
`failed`	Falló el proceso o el PDF no tiene texto seleccionable (0 fragmentos). El campo `error` guarda la causa.

> Detalles de robustez

- **Idempotencia:** la restricción `(owner_id, sha256)` evita duplicados y permite reintentar.
- **Documentos interrumpidos:** al arrancar la API se marcan como `failed` los que quedaron a medias por un reinicio.

- **Seguimiento en la interfaz:** el frontend consulta la lista cada 3 s hasta que todos los documentos llegan a un estado final.
- **Borrado:** eliminar un documento propio elimina también sus fragmentos gracias al `CASCADE`.
- **Libros completos:** los libros grandes se cargan con el script `scripts/ingest_corpus.py`; la subida web se reserva para extractos de unas 5 a 30 páginas.

#10 Estrategias implementadas para evitar alucinaciones

El enunciado pide al menos dos estrategias; DataTutor combina **cinco**, que actúan en distintos puntos del grafo.

#	Estrategia	Dónde actúa	Efecto
1	**Umbral de similitud** (`SIM_LOW` 0,50 y `SIM_THRESHOLD` 0,65)	`grade_context`	Si lo recuperado no se parece lo bastante a la pregunta, no se intenta responder.
2	**Nodo de validación de suficiencia** (juez LLM + respaldo léxico)	`grade_context`	Verifica que el contexto realmente contiene lo necesario antes de generar texto.
3	**Prompt restrictivo**	`generate_answer`	Obliga a responder solo con el contexto numerado, sin conocimiento externo; temperatura 0.
4	**Rechazo explícito con mensaje fijo**	`refuse`	Si no hay evidencia, el sistema lo dice en lugar de improvisar y sugiere cargar documentos.
5	**Fuentes visibles en cada respuesta**	API y frontend	Se muestran documento, página, similitud y fragmento para que el usuario pueda verificar.

> Protecciones adicionales

- **Fuera de dominio sin consultar la base:** las preguntas ajenas al tema (por ejemplo, recetas) se rechazan antes de la recuperación.
- **Juez conservador:** una salida inválida del juez cuenta como «no suficiente».
- **Aclaración en lugar de adivinar:** las consultas de 4 palabras o menos que no alcanzan evidencia piden precisión.
- **Aislamiento por usuario:** el agente sólo recupera documentos propios y globales.

> Limitaciones reconocidas

> Las respuestas **no muestran citas '[n]' en línea**, aunque el prompt las pide; las fuentes aparecen al final de la respuesta. Eso no demuestra por sí solo que cada afirmación este respaldada.
>
> El modelo de 3B puede usar conocimiento propio fuera del contexto. Se revisó una respuesta sobre formas normales que contenía definiciones no evidentes en los fragmentos recuperados.

#11 Seguridad y autenticación

> Autenticación con Keycloak

La autenticación la gestiona **Keycloak 24** con OpenID Connect, usando el flujo Authorization Code con PKCE (S256). El backend nunca maneja contraseñas.

1. El frontend redirige al usuario a Keycloak (realm `datatutor`, cliente público `datatutor-web`).
2. Keycloak autentica al usuario y devuelve un código que el frontend canjea por un token de acceso.
3. En cada petición, el frontend envía `Authorization: Bearer <token>`.
4. La API valida el token y solo entonces atiende la solicitud.

> Validación del JWT en la API

Comprobación	Detalle
Firma	RS256, con las claves públicas de Keycloak (JWKS) obtenidas por `127.0.0.1`.
Emisor (`iss`)	`http://localhost:8080/realms/datatutor`.
Audiencia (`aud`)	`datatutor-web`.
Vigencia (`exp`)	El token vencido se rechaza con 401.
Identidad	El claim `sub` se usa como `owner_id` en todas las consultas.

> Medidas de protección

- **Ningún endpoint sin autenticar,** salvo `GET /health`.
- **Aislamiento entre usuarios:** conversaciones, mensajes y documentos se filtran por `owner_id` en cada consulta; un recurso ajeno responde 404.
- **Cliente público sin Direct Access Grants** para la aplicación web. Un cliente aparte (`datatutor-swagger`) existe solo para scripts de consola.
- **Token en memoria:** `keycloak-js` lo conserva en memoria, no en `localStorage`, y lo renueva de forma preventiva.
- **Subidas validadas:** tamaño máximo de 30 MB, extensión y tipo real; el contenido nunca se ejecuta.
- **CORS limitado** al origen del frontend (`http://localhost:5173`).
- **Servicios locales:** Keycloak, PostgreSQL y Ollama escuchan solo en la máquina local.
- **Secretos fuera del código:** las credenciales están en `.env`, que no se versiona; el repositorio incluye solo `.env.example`. Los volcados de base de datos (`*.dump`) y los PDF del corpus tampoco se suben.

> Cadena de suministro

Se detectaron campañas activas de paquetes maliciosos en npm. Por eso el frontend se instaló con nombres exactos y `npm install --ignore-scripts`, se revisó el `package-lock.json` y se ejecutó `npm audit signatures` (25 paquetes con firma verificada y 15 con atestaciones verificadas). Las dependencias del backend quedan fijadas en `uv.lock`.

> Verificación

- 401 sin token y con token manipulado (script de la Fase 3).
- Aislamiento entre usuarios: un recurso ajeno devuelve 404 (script de la Fase 3 y prueba automática en `pytest`).
- Prueba automática de que los endpoints protegidos rechazan a usuarios anónimos.

#12 Despliegue

> **Estado al 17 de octubre de 2026.** El sistema completo funciona de forma local. El despliegue público es opcional según el enunciado cuando el campus lo bloquea y aún no se había realizado al redactar este documento. Actualizar esta sección con las URLs si se completa antes de la entrega.

> Entorno local

Docker Compose levanta PostgreSQL y Keycloak. La API, el frontend y Ollama corren directamente en el portátil. Esta configuración permite una demostración presencial sin internet.

> Orden de arranque

1. Abrir Docker Desktop e iniciar los contenedores `datatutor-postgres` y `datatutor-keycloak`.
2. Iniciar Ollama en CPU con `ollama serve`; `ollama list` debe mostrar `qwen2.5:3b` y `bge-m3`.
3. Desde `backend/`, iniciar la API con PyCharm o `uv run uvicorn app.main:app --reload`.
4. Desde `frontend/`, ejecutar `npm run dev` y abrir `http://localhost:5173`.

> Variables de entorno

Los valores viven en `.env` (backend) y `frontend/.env` (solo variables públicas con prefijo `VITE_`); ambos tienen su plantilla `.env.example`.

Grupo	Variables
Base de datos	`DATABASE_URL`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`
Modelo local	`OLLAMA_URL`, `OLLAMA_KEEP_ALIVE`, `CHAT_MODEL`, `EMBED_MODEL`, `EMBED_DIM`, `NUM_CTX`, `TEMPERATURE`, `NUM_PREDICT`
RAG	`CHUNK_SIZE`, `CHUNK_OVERLAP`, `TOP_K`, `SIM_THRESHOLD`, `SIM_LOW`, `MAX_UPLOAD_MB`

Grupo	Variables
Keycloak	`KEYCLOAK_URL`, `KEYCLOAK_JWKS_URL`, `KEYCLOAK_REALM`, `KEYCLOAK_CLIENT_ID`, `KEYCLOAK_AUDIENCE`, `KC_ADMIN_USER`, `KC_ADMIN_PASSWORD`
API	`CORS_ORIGINS`
Frontend	`VITE_API_BASE_URL`, `VITE_KEYCLOAK_URL`, `VITE_KEYCLOAK_REALM`, `VITE_KEYCLOAK_CLIENT_ID`, `VITE_MAX_UPLOAD_MB`

> Despliegue público (Fase 7)

El plan para obtener URLs públicas es publicar el frontend en Vercel o Netlify y exponer el backend y Keycloak del portátil mediante túneles con ngrok. PostgreSQL permanece local, ya que solo la API se comunica con él.

El principal reto es la identidad: un navegador externo intentaría conectarse a `localhost:8080` para Keycloak.

Elemento	URL
Frontend público	[https://data-tutor.vercel.app/]
Backend público	[https://datatutor.onrender.com]

#13 Evidencias de funcionamiento

> Pruebas por fase

Fase	Evidencia
0 · Entorno	`verificar_entorno.py`: PostgreSQL + pgvector, JWKS de Keycloak y Ollama con `qwen2.5:3b` y `bge-m3`.
1–2 · Ingesta y agente	Re-embedding de 57 fragmentos (coseno 1,000000, sin nulos, 1024 dim.); recuperación con puntajes de 0,7172 (inglés) y 0,6825 (español); CLI de chat con tres casos: pregunta respondible, sin soporte y saludo.
3 · API	`test_api.py`: JWT válido, 401 sin token y con token manipulado, `POST /chat` 200 y aislamiento 404 entre usuarios.
4 · Documentos	`test_documents.py`: 202, estado `ready`, listado 200, duplicado 409, validación de embeddings, borrado 204 y cascada.
5 · Frontend	`pytest` 5 pruebas aprobadas; `vite build` correcto; login OIDC + PKCE, historial, seguimiento de documentos y rechazo de PDF sin texto verificados en la interfaz.

> Resultado de pytest

```
collected 5 items

test_health_and_openapi_expose_all_endpoints ..................... PASSED
test_protected_endpoints_reject_anonymous ........................ PASSED
test_database_foreign_keys_enforce_sql_on_delete_cascade ......... PASSED
test_chat_503_returns_conversation_id_to_avoid_duplicates ........ PASSED
test_conversation_lifecycle_and_cross_user_isolation ............. PASSED

5 passed in 1.82s   (Python 3.12.7, pytest 9.1.1)
```

> Pruebas de chat

Pregunta	Decisión	Puntaje	Latencia
¿Qué es la normalización y para qué sirven las formas normales?	`answered`	0,706	33,0 s
¿Qué son la segunda y la tercera forma normal?	`answered`	0,665	32,1 s
¿Cuál es la mejor receta para preparar sushi?	`rejected` (fuera de dominio)	0,000	1,3 s
¿Cómo se configura un clúster de Apache Kafka con ZooKeeper en producción?	`rejected`	0,542	16,6 s
¿Qué es una automatización?	`clarify`	0,607	13,9 s
¿Qué es una automatización en procesos de machine learning?	`rejected`	0,550	12,6 s

Los rechazos de sushi (sin consultar la base) y Kafka (sin evidencia en el corpus) son correctos: demuestran que el agente no inventa. La última pregunta podría ser un falso negativo, porque hay un documento de inteligencia artificial indexado; está pendiente de revisión.

> Capturas de pantalla

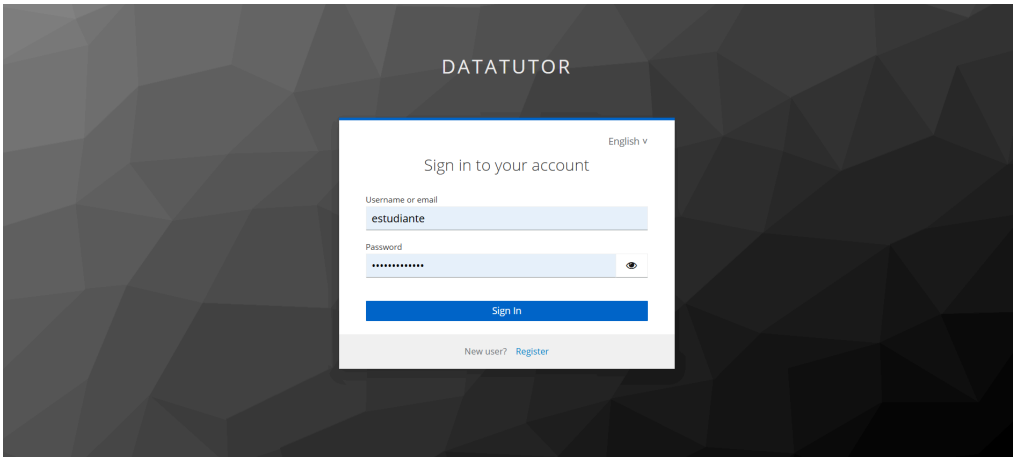


Figura 3. Login

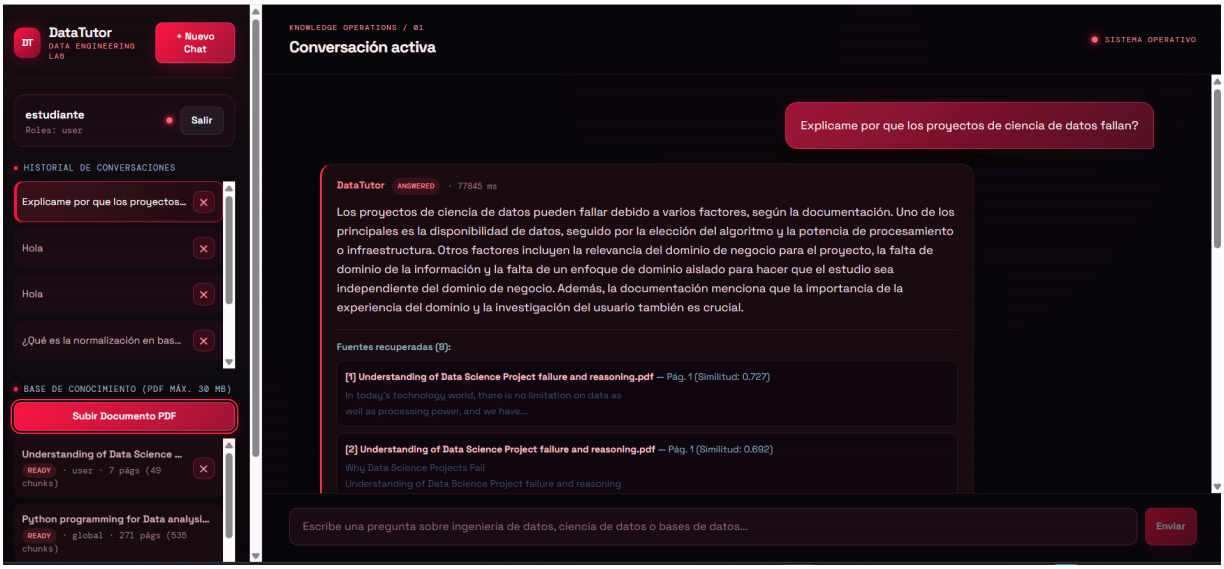


Figura 4. Petición exitosa

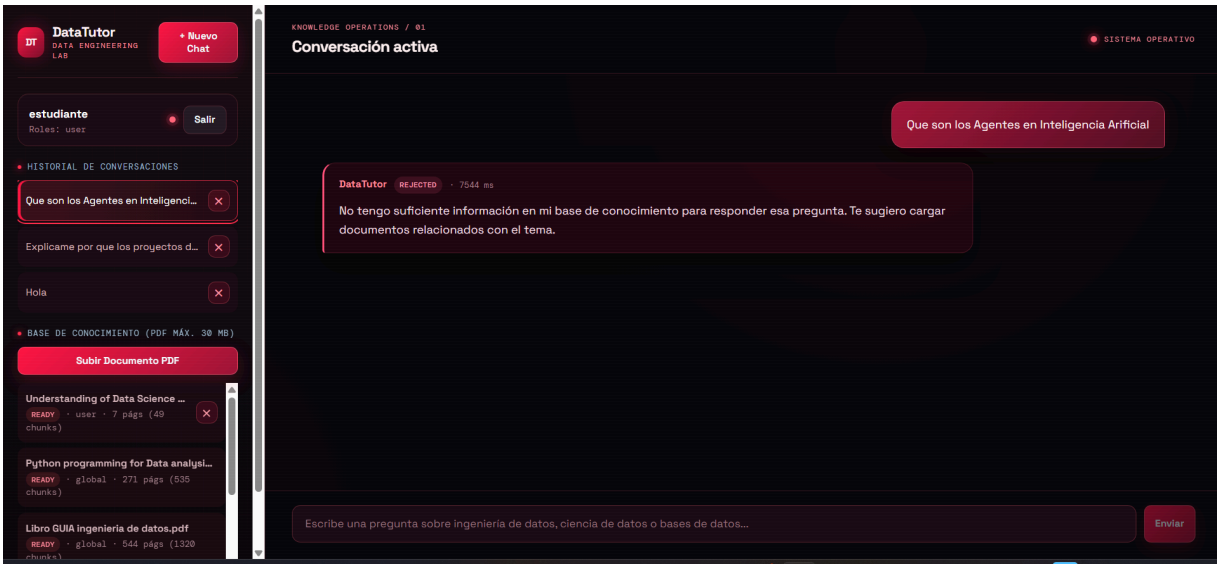


Figura 5. Petición rechazada

> Evaluación

El proyecto incluye un conjunto de evaluación (`eval/questions.jsonl`, de 30 a 40 preguntas: respondibles, fuera de dominio y plausibles pero ausentes) para calibrar los umbrales y medir aciertos, rechazos correctos, falsos rechazos y latencia.

#14 Problemas encontrados y conclusiones

> Problemas y soluciones

Problema	Causa	Solución
Ollama fallaba al generar	La GPU del equipo está dañada y la inferencia fallaba con un error CUDA	Ejecutar solo en CPU con `CUDA_VISIBLE_DEVICES=-1` y `OLLAMA_LLM_LIBRARY=cpu_avx2`
Búsqueda irrelevante con documentos ya cargados	Los embeddings guardados no coincidían con el modelo actual	Respaldo de la base y re-embedding en sitio de los 57 fragmentos
Normalización rechazada con puntaje 0,706	Falso negativo del juez de 3B	Respaldo léxico en `grade_context`
El juez con JSON inválido daba contexto válido	El `except` devolvía `context_ok = True`	Ahora devuelve `False`
`clarify` para preguntas claras sin evidencia	Decisión basada solo en el puntaje	`clarify` solo para consultas de 4 palabras o menos; el resto se rechaza
Documentos atascados en `processing`	El modo `--reload` interrumpía las tareas en segundo plano	`fail_interrupted_documents` al arrancar; libros grandes por script
PDF sin texto marcado como `ready`	La extracción devolvía 0 fragmentos	Se marca `failed`
Historial sin texto en la interfaz	El esquema `MessageItem` no incluía `content`	Campo añadido en `schemas/chat.py`
Puerto 5432 ocupado	El servicio PostgreSQL de Windows ocupaba el puerto	Servicio detenido y deshabilitado
Errores de autenticación con PostgreSQL	Credenciales distintas y colisión con el PostgreSQL nativo	`ALTER USER` y sincronización de `.env` con `settings.py`
El CLI ejecutaba el grafo dos veces	Dos bucles sobre el mismo `stream`	Un solo `stream` con `stream_mode="updates"`
`npm install` colgado 30 minutos	Red universitaria	Cambio de red; terminó en 14 s

> Limitaciones abiertas

- **Latencia:** de 12 a 33 s por consulta, al ejecutar varias llamadas al modelo en CPU.
- **Sin citas en línea** y posible uso de conocimiento propio del modelo de 3B.
- **Reescritura de consulta:** `rewrite_query` produce consultas casi idénticas a la original, por lo que el segundo intento recupera prácticamente lo mismo.
- **Visibilidad del corpus:** los libros ingeridos como `user` solo los ve su propietario; falta decidir si se convierten a `global`.

- **Pruebas pendientes:** respaldo léxico sin pruebas unitarias propias, calibración con el conjunto de evaluación y CI.
- **Datos de prueba:** las pruebas automáticas dejan filas en la base de desarrollo.

> Conclusiones

DataTutor cumple el objetivo central del trabajo: un agente construido en código con LangChain y LangGraph que responde a partir de una base de conocimiento propia, con un modelo local, autenticación real y rechazo explícito cuando no tiene sustento. Las pruebas muestran que el sistema distingue entre preguntas que puede responder y las que no, como las de sushi o Kafka.

El proyecto dejó aprendizajes claros. Un modelo pequeño es suficiente para un asistente verificable si el diseño lo respalda: el valor no está en el tamaño del modelo, sino en el grafo que lo rodea, con umbrales, validación de suficiencia, prompt restrictivo y rechazo. También mostró que las heurísticas simples, como el respaldo léxico, pueden corregir fallos de un modelo pequeño, siempre que se documenten sus límites.

La arquitectura en capas, la inyección de dependencias y las pruebas con dobles permitieron cambiar partes del sistema sin romper el resto. Quedan por resolver la latencia, las citas en línea y la calibración con el conjunto de evaluación, y esas tareas están identificadas como trabajo futuro.